NLP Text Preprocessing

1. Setting up NLTK

NLTK (Natural Language Toolkit) is one of the most popular Python libraries for NLP. It was developed at the University of Pennsylvania and provides:

- Interfaces for 50+ corpora and lexical resources (e.g., WordNet)
- Tools for tokenization, stemming, tagging, parsing, and more
- Pre-trained models ready to use

> WordNet is a lexical database of semantic relations between words in 200+ languages.

```
%pip install nltk
```

After importing NLTK, you may want to download additional resources like corpora or models depending on your requirements. NLTK provides a convenient way to download these resources using the nltk.download() function.

To download all the available resources at once, you can run: Note: Alternatively, you can choose specific resources to download by replacing 'all' with their respective identifiers.

```
#nltk.download('all')
```

```
import nltk
import re
import string

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\dell\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\dell\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\dell\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\dell\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]     C:\Users\dell\AppData\Roaming\nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\dell\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\dell\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
True
```

2. Tokenization

Tokenization is the process of splitting text into smaller units called *tokens*.

There are several types:

- **Word Tokenization** → splits text into individual words
- **Sentence Tokenization** → splits text into sentences
- **WordPunctTokenizer** → splits on whitespace and punctuation
- **RegexpTokenizer** → custom splitting using regular expressions

Manual (split on white spaces)

```
text_simple = """Paperspace is built for the future where hardware and software are
inextricably linked. Our tools provide a seamless abstraction layer that
radically simplifies access to accelerated computing. The Paperspace
stack removes costly distractions, enabling individuals & organizations
to focus on what matters."""

text_nlp = "Tokenization is an important step in Natural Language Processing (NLP). It breaks down text into smaller units

print("Sample text (simple):")
print(text_simple)
print("Sample text (NLP):")
print(text_nlp)
```

```
Sample text (simple):
Paperspace is built for the future where hardware and software are
inextricably linked. Our tools provide a seamless abstraction layer that
radically simplifies access to accelerated computing. The Paperspace
stack removes costly distractions, enabling individuals & organizations
to focus on what matters.
Sample text (NLP):
Tokenization is an important step in Natural Language Processing (NLP). It breaks down text into smaller units called tokens
```

```
text_nlp.split()
```

```
['Tokenization',
 'is',
 'an',
 'important',
 'step',
 'in',
 'Natural',
 'Language',
 'Processing',
 '(NLP).',
 'It',
 'breaks',
 'down',
 'text',
 'into',
 'smaller',
 'units',
 'called',
 'tokens.',
 'These',
 'tokens',
 'can',
 'be',
 'words,',
 'sentences,',
 'or',
 'even',
 'characters.']
```

2.1 Word Tokenization

```
from nltk.tokenize import word_tokenize

text_clean = text_simple.lower()
text_clean = re.sub(r'[^\w\s]', '', text_clean)

print("After cleaning:", text_clean)
```

```
After cleaning: paperspace is built for the future where hardware and software are
inextricably linked our tools provide a seamless abstraction layer that
radically simplifies access to accelerated computing the paperspace
stack removes costly distractions enabling individuals  organizations
to focus on what matters
```

```
word_tokens = word_tokenize(text_clean)

print(f"Tokens ({len(word_tokens)}):", word_tokens)
```

```
Tokens (41): ['paperspace', 'is', 'built', 'for', 'the', 'future', 'where', 'hardware', 'and', 'software', 'are', 'inextrica
```

2.2 Sentence Tokenization

```
from nltk.tokenize import sent_tokenize

sentences = sent_tokenize(text_nlp)
print(sentences)
```

```
['Tokenization is an important step in Natural Language Processing (NLP).', 'It breaks down text into smaller units called t
```

2.3 WordPunctTokenizer

Splits on **whitespace AND punctuation** — keeps punctuation as separate tokens.

```
from nltk.tokenize import WordPunctTokenizer

s = "Good muffins cost $3.88\nin New York.  Please buy me\ntwo of them.\n\nThanks."

wp_tokens = WordPunctTokenizer().tokenize(s)
print(wp_tokens)
```

```
['Good', 'muffins', 'cost', '$', '3', '.', '88', 'in', 'New', 'York', '.', 'Please', 'buy', 'me', 'two', 'of', 'them', '.',
```

2.4 RegexpTokenizer

Lets you define **custom rules** using regular expressions.

Pattern	Behavior
`[\w]+`	Keep alphanumeric + underscore
`[a-zA-Z0-9_]+`	Explicit alphanumeric
`\s+` with `gaps=True`	Split on whitespace (keep punctuation attached)

```
from nltk.tokenize import RegexpTokenizer

s = "Good muffins cost $3.88\nin New York.  Please buy me\ntwo of them.\n\nThanks."

tokenizer1 = RegexpTokenizer(r"[\w]+")
print("[\\w]+ :", tokenizer1.tokenize(s))

tokenizer2 = RegexpTokenizer(r"[a-zA-Z0-9_]+")
print("[a-zA-Z0-9_]+ :", tokenizer2.tokenize(s))

tokenizer3 = RegexpTokenizer(r'\s+', gaps=True)
print("\\s+ gaps=True :", tokenizer3.tokenize(s))
```

```
[\w]+ : ['Good', 'muffins', 'cost', '3', '88', 'in', 'New', 'York', 'Please', 'buy', 'me', 'two', 'of', 'them', 'Thanks']
[a-zA-Z0-9_]+ : ['Good', 'muffins', 'cost', '3', '88', 'in', 'New', 'York', 'Please', 'buy', 'me', 'two', 'of', 'them', 'Tha
\s+ gaps=True : ['Good', 'muffins', 'cost', '$3.88', 'in', 'New', 'York.', 'Please', 'buy', 'me', 'two', 'of', 'them.', 'Tha
```

2.5 TreebankWordTokenizer

Based on the **Penn Treebank** conventions. It's great for handling **contractions** — splits `don't` → `do`, `n't` and `can't` → `ca`, `n't`.

```
from nltk.tokenize import TreebankWordTokenizer

text_contractions = "What you can't want to be done to yourself, don't do to others..."

tokenizer = TreebankWordTokenizer()
result = tokenizer.tokenize(text_contractions)

print(result)
```

```
['What', 'you', 'ca', "n't", 'want', 'to', 'be', 'done', 'to', 'yourself', ',', 'do', "n't", 'do', 'to', 'others', '...']
```

2.6 TweetTokenizer

Specifically designed for **social media text**. It correctly handles:

- **Emojis** 😄👌 (kept as tokens)
- **Hashtags** #Hope (kept together)
- **Mentions** @user
- **Contractions** like `Don't`

```
from nltk.tokenize import TweetTokenizer

tweet = "Don't take cryptocurrency advice from people on Twitter 😂👌"

tokenizer = TweetTokenizer()
result = tokenizer.tokenize(tweet)

print(result)
```

```
["Don't", 'take', 'cryptocurrency', 'advice', 'from', 'people', 'on', 'Twitter', '😂', '👌']
```

2.7 MWETokenizer (Multi-Word Expression)

MWETokenizer lets you define **multi-word expressions** that should be treated as **a single token**.

Example: `Hunger Games` → `Hunger_Games` (one token instead of two)

This is very useful for named entities, technical terms, and fixed expressions.

```
from nltk.tokenize import MWETokenizer

text_mwe = "Hope, is the only thing stronger than fear! Hunger Games #Hope"

tokenizer_basic = MWETokenizer()
result_basic = tokenizer_basic.tokenize(word_tokenize(text_mwe))
print("Without MWE:", result_basic)

tokenizer_mwe = MWETokenizer()
tokenizer_mwe.add_mwe(('Hunger', 'Games'))
result_mwe = tokenizer_mwe.tokenize(word_tokenize(text_mwe))
print("With MWE   :", result_mwe)
```

```
Without MWE: ['Hope', ',', 'is', 'the', 'only', 'thing', 'stronger', 'than', 'fear', '!', 'Hunger', 'Games', '#', 'Hope']
With MWE   : ['Hope', ',', 'is', 'the', 'only', 'thing', 'stronger', 'than', 'fear', '!', 'Hunger_Games', '#', 'Hope']
```

```
tokenizer_ne = MWETokenizer()
tokenizer_ne.add_mwe(('New', 'York'))
tokenizer_ne.add_mwe(('Natural', 'Language', 'Processing'))
tokenizer_ne.add_mwe(('Machine', 'Learning'))

text_ne = "Natural Language Processing and Machine Learning are popular in New York."
tokens_ne = tokenizer_ne.tokenize(word_tokenize(text_ne))

print(tokens_ne)
```

```
['Natural_Language_Processing', 'and', 'Machine_Learning', 'are', 'popular', 'in', 'New_York', '.']
```

3. Stop Word Removal

Stop words are common words (like *is, the, in, of*) that:

- Appear very frequently
- Usually carry **no meaningful information**
- Can be safely removed to reduce noise

NLTK provides a built-in stop word list for many languages.

```
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))

print(f"Total stop words: {len(stop_words)}")
print("Sample:", sorted(stop_words)[:10])
```

```
Total stop words: 198
Sample: ['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an']
```

```
word_tokens_cleaned = [word for word in word_tokens if word.casefold() not in stop_words] # not Not

print(f"Before: {len(word_tokens)} tokens")
print(f"After : {len(word_tokens_cleaned)} tokens")
print(word_tokens_cleaned)
```

```
Before: 41 tokens
After : 27 tokens
['paperspace', 'built', 'future', 'hardware', 'software', 'inextricably', 'linked', 'tools', 'provide', 'seamless', 'abstrac
```

4. Text Normalization: Stemming & Lemmatization

Users express the same concept in many ways:
`lovely`, `luv`, `loveee` → all essentially mean **love**

Text normalization helps **reduce these variations** to a common base form.

Two main approaches:

	Stemming	Lemmatization
How it works	Rule-based (removes suffixes)	Looks up word in WordNet
Output	May not be a real word	Always a real word
Speed	Faster	Slower
Example	`studies` → `studi`	`studies` → `study`

4.1 Stemming with PorterStemmer

```
from nltk.stem.porter import PorterStemmer

stemmer = PorterStemmer()

words = ['caresses', 'flies', 'dies', 'mules', 'denied',
         'died', 'agreed', 'owned', 'humbled', 'sized',
         'meeting', 'stating', 'siezing', 'itemization',
         'sensational', 'traditional', 'reference', 'colonizer', 'plotted']

for word in words:
    print(f" {stemmer.stem(word)}")
```

```
 caress
 fli
 die
 mule
 deni
 die
 agre
 own
 humbl
 size
 meet
 state
 siez
 item
 sensat
 tradit
 refer
 colon
 plot
```

```
stemmed_tokens = [stemmer.stem(token) for token in word_tokens_cleaned]

print(stemmed_tokens)
```

```
['paperspac', 'built', 'futur', 'hardwar', 'softwar', 'inextric', 'link', 'tool', 'provid', 'seamless', 'abstract', 'layer',
```

4.2 Snowball Stemmer — Multi-language Support

```
from nltk.stem.snowball import SnowballStemmer

print(", ".join(SnowballStemmer.languages))
```

```
arabic, danish, dutch, english, finnish, french, german, hungarian, italian, norwegian, porter, portuguese, romanian, russia
```

```
#arabic example
ar_stemmer = SnowballStemmer("arabic")
arabic_text = "الجو سماؤه صافية"
print(f"Arabic input:  {arabic_text}")
print(f"After stemming: {ar_stemmer.stem(arabic_text)}")
```

```
Arabic input:  الجو سماؤه صافية
After stemming: جو سماوه صاف
```

4.3 Lemmatization with WordNetLemmatizer

```
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

# Examples (pos: n=noun, v=verb, a=adjective, r=adverb)
examples = [
    ("rocks",  "n"),
    ("corpora", "n"),
    ("better",  "a"),
    ("running", "v"),
    ("studies", "v"),
]

for word, pos in examples:
    print(f"{lemmatizer.lemmatize(word, pos=pos)}")
```

```
rock
corpus
good
run
study
```

```
lemmatized_tokens = [lemmatizer.lemmatize(token) for token in word_tokens_cleaned]

print(lemmatized_tokens)
```

```
['paperspace', 'built', 'future', 'hardware', 'software', 'inextricably', 'linked', 'tool', 'provide', 'seamless', 'abstract
```

4.4 Stemming vs Lemmatization

> **Key insight:** Stemming can produce non-real words (e.g., `studi`), while lemmatization always returns a valid dictionary word. However, stemming is faster.

```
test_words = ['studies', 'running', 'better', 'meeting', 'sensational', 'denied']
for word in test_words:
    print(f"{stemmer.stem(word)} : {lemmatizer.lemmatize(word)}")
```

```
studi : study
run : running
better : better
meet : meeting
sensat : sensational
deni : denied
```

5. Handling Special Characters & Punctuation

Text data often contains punctuation, symbols, and special characters that need to be removed or handled before further processing.

```
tokens_raw = word_tokenize(text_nlp)
special_chars = set(string.punctuation)

filtered_no_punct = [token for token in tokens_raw if token not in special_chars]

print("Raw   :", tokens_raw)
print("Clean :", filtered_no_punct)
```

```
Raw   : ['Tokenization', 'is', 'an', 'important', 'step', 'in', 'Natural', 'Language', 'Processing', '(', 'NLP', ')', '.', '
Clean : ['Tokenization', 'is', 'an', 'important', 'step', 'in', 'Natural', 'Language', 'Processing', 'NLP', 'It', 'breaks',
```

```
text_example = "Hello!!! This is NLP #2024. Visit https://example.com for more info."

text_example = re.sub(r'http\S+|www\S+', '', text_example)

text_example = re.sub(r'[^a-zA-Z\s]', '', text_example)

print("Text  :", text_example)
print("Tokens:", word_tokenize(text_example))
```

```
Text  : Hello This is NLP  Visit  for more info
Tokens: ['Hello', 'This', 'is', 'NLP', 'Visit', 'for', 'more', 'info']
```

6. Part-of-Speech (POS) Tagging

POS tagging labels each word with its grammatical role:

Tag	Meaning	Example
NN	Noun, singular	dog, city
NNS	Noun, plural	dogs, cities
NNP	Proper noun	London, John
VB	Verb, base form	run, eat
VBZ	Verb, 3rd person	runs, eats
VBG	Verb, gerund	running, eating
JJ	Adjective	big, happy
RB	Adverb	quickly, very
IN	Preposition	in, of, for
DT	Determiner	the, a, an

```
text_pos = "NLTK provides powerful tools for performing POS tagging"
tokens_pos = word_tokenize(text_pos)

pos_tags = nltk.pos_tag(tokens_pos)

for token, tag in pos_tags:
    print(f"{token} : {tag}")
```

```
NLTK : NNP
provides : VBZ
powerful : JJ
tools : NNS
for : IN
performing : VBG
POS : NNP
tagging : NN
```

```
pos_tag_result = nltk.pos_tag(word_tokens_cleaned)

print(pos_tag_result)
```

```
[('paperspace', 'NN'), ('built', 'VBN'), ('future', 'JJ'), ('hardware', 'NN'), ('software', 'NN'), ('inextricably', 'RB'), (
```

7. Word Frequency Distribution

Frequency distribution shows how often each word appears in the text.

This is useful for:

- Understanding the most important terms
- Feature engineering for text classification
- Detecting noise or irrelevant words

```
from nltk import FreqDist

freq_dist = FreqDist(word_tokens_cleaned)

for word, count in freq_dist.most_common(10):
    print(f"{word} = {count}")
```

```
paperspace = 2
built = 1
future = 1
hardware = 1
software = 1
inextricably = 1
linked = 1
tools = 1
provide = 1
seamless = 1
```

```
freq_lemmatized = FreqDist(lemmatized_tokens)

print(freq_lemmatized.most_common(10))
```

```
[('paperspace', 2), ('built', 1), ('future', 1), ('hardware', 1), ('software', 1), ('inextricably', 1), ('linked', 1), ('toc
```

8. Named Entity Recognition (NER)

NER identifies and classifies real-world entities in text:

Entity Type	Examples
PERSON	Steve Jobs, Elon Musk
ORGANIZATION	Apple Inc., Google
GPE (Location)	California, New York
DATE	January 2024

```
import nltk
nltk.download('maxent_ne_chunker', quiet=True)
nltk.download('maxent_ne_chunker_tab', quiet=True)
nltk.download('words', quiet=True)

from nltk import ne_chunk
from nltk.tokenize import word_tokenize

text = "Inc. was founded by Steve Jobs and Steve Wozniak. Its headquarters are in Cupertino, California."

tokens  = word_tokenize(text)
pos     = nltk.pos_tag(tokens)
ner     = ne_chunk(pos)

for chunk in ner:
    if hasattr(chunk, 'label'): #(name,label)
        entity = ' '.join(c[0] for c in chunk)
        print(f"{chunk.label()} ----- {entity}")
```

```
PERSON ----- Steve Jobs
PERSON ----- Steve Wozniak
GPE ----- Cupertino
GPE ----- California
```